Introduction to Credit Risk and Applications in Python

Prof. Dr. Eva Lütkebohmert-Holtz
M.Sc. Hongyi Shen

April 16, 2026

Department of Quantitative Finance
Faculty of Economics and Behavioral Sciences
University of Freiburg

Lecture Outline

1. Introduction
2. Python
2.1 Python: Setup
2.2 Git: Setup
2.3 Workflow
2.4 Python: Programming

Introduction

The Illusion of Stability

- Most of the time, portfolios appear stable.
- Observed default rates are low.
- Losses cluster around small expected values.

Why Study Credit Risk?

If Defaults Are Rare, Where Does Systemic Risk Come From?

- **Dependence —** Defaults do not occur independently.
- **Factor structure —** Common economic drivers create clustering.
- **Model assumptions —** Distributional choices shape outcomes.
- **Tail amplification —** Small structural effects become large in extremes.

Dependence and Tail Amplification: A Simulation

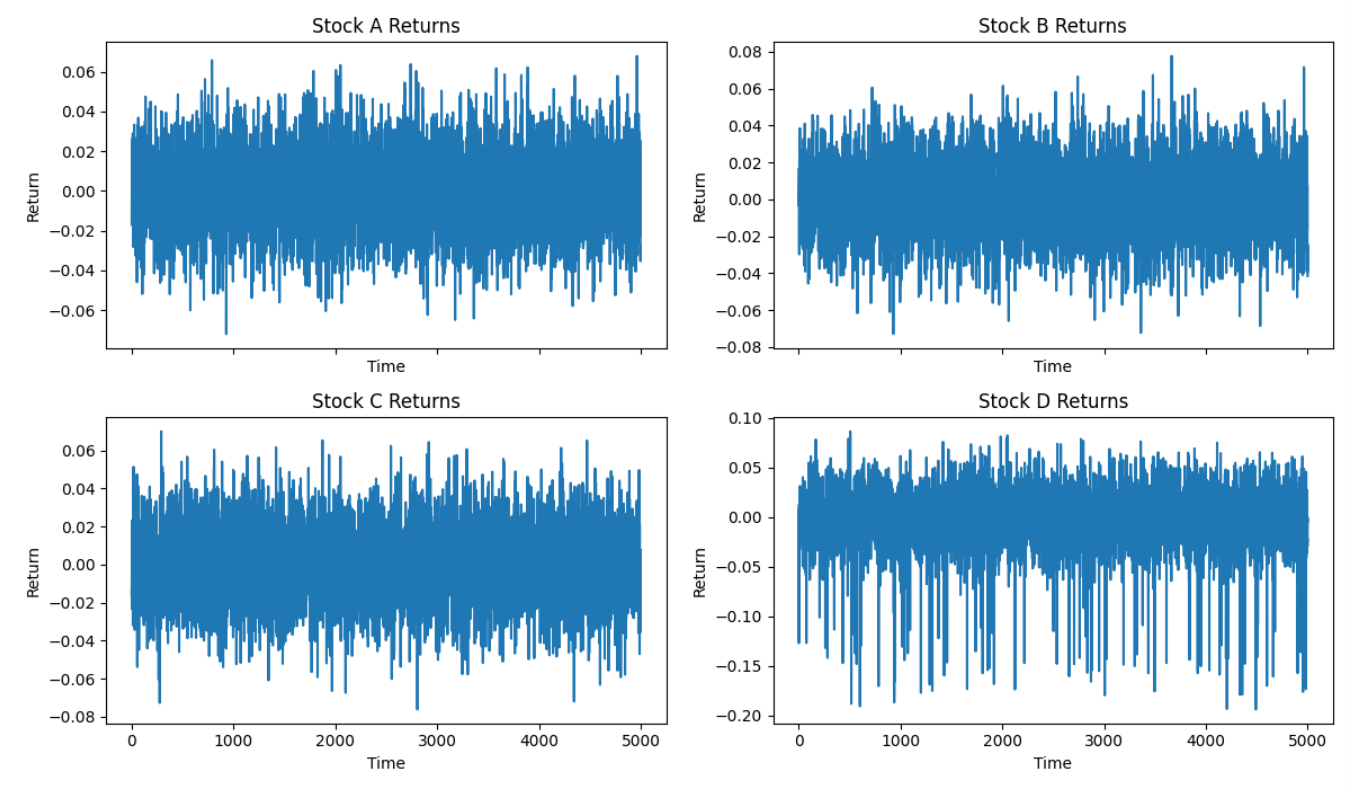

Figure 1: Simulated Multivariate Returns with Correlation and Tail Shock

Uncomfortable Facts

- **Fact 1 — Credit risk is usually invisible in calm periods.**
- **Fact 2 — Correlation drives extreme portfolio losses.**
- **Fact 3 — Model assumptions determine measured capital.**

From Stability to Systemic Risk

- Risk is not about average losses.
- Risk is about rare joint events.
- Banks do not fail because a single borrower defaults.
- Banks fail when many exposures move together.
- Regulatory capital is determined by assumptions about dependence and tail behavior.

What This Course Is About (Modeling View)

- **1. Modeling individual default.** Describe whether a firm defaults as a random outcome driven by its underlying financial condition.

- **2. From individual firms to a portfolio.** Aggregate many firms to understand how individual defaults translate into total portfolio losses.

- **3. Comovement and systemic risk.** Capture how firms are jointly affected by common economic factors, leading to correlated defaults.

- **4. Risk management and capital.** Quantify potential losses and determine how much capital is needed to absorb extreme outcomes.

Credit Risk Measurement

Python

Python in This Course

Python for Credit Risk Modelling

In this course, Python is used as a **research environment**, not as a programming course.

Focus:

- implementing quantitative models
- running simulations
- analysing risk distributions
- producing reproducible results

Goal: Use programming as a tool for financial modelling.

Course Philosophy

Modern coding tools and AI can assist with writing code.

However, successful quantitative work still requires:

- understanding model structure
- debugging incorrect implementations
- designing simulations
- interpreting statistical outputs

Key principle:

Programming is a **tool for modelling and analysis**, not an end in itself.

Development Environment Used in This Course

In this course we standardize the working environment to simplify collaboration.

Two main tools will be used:

- **PyCharm** as the primary Python development environment
- **GitHub** as the collaboration and version control platform

Python: Setup

Installing Python

Python must be installed before setting up the development environment.

Step 1: Download Python

- Visit: `https://www.python.org`
- Download the latest stable version of Python
- Recommended: Python 3.x

Step 2: Install Python

- Run the installer
- Select **Add Python to PATH**
- Complete the installation

Verification

Installing PyCharm

PyCharm will be used as the main development environment.

Step 1: Download PyCharm

- Visit: `https://www.jetbrains.com/pycharm`
- Download **PyCharm Community Edition**

Step 2: Install the IDE

Step 3: Create a Project

- Open PyCharm
- Click **New Project**
- Select the installed Python interpreter

Git: Setup

Installing Git

Git is required to interact with GitHub repositories.

Step 1: Download Git

- Visit: `https://git-scm.com`
- Download the installer for your operating system

Step 2: Install Git

- Run the installer
- Accept default configuration options

Verification

```
git --version
```

Creating a GitHub Account

GitHub will be used for collaboration.

Step 1: Create an Account

- Visit: `https://github.com`
- Click **Sign Up**
- Create a username and password

Step 2: Configure Git

After installing Git, configure your identity:

```
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"
```

Workflow

Structured Projects

Typical structure:

```
credit-risk-project/

 data/
 src/
    simulation.py
    models.py
    utils.py
 notebooks/
 results/
 requirements.txt
 README.md
```

Principle: Separate data, code, analysis, and results.

Virtual Environments

Python environments isolate project dependencies.

Benefits:

- consistent package versions
- fewer compatibility issues
- easier replication of results

Common tools:

- `venv`
- `conda`

Dependencies are stored in

`requirements.txt`

Why Version Control?

Without version control:

- files get overwritten
- changes are difficult to track
- collaboration becomes unreliable

Version control systems record:

- who changed code
- what changed
- when it changed

Standard tool: **Git**

Basic Git Workflow

Typical development workflow:

1. Clone the repository
2. Create a new branch
3. Implement a feature
4. Commit changes
5. Submit a pull request

Benefits:

- changes are documented
- work can be reviewed
- errors can be reverted

Collaboration on GitHub

GitHub provides infrastructure for team development.

Main collaboration tools:

- pull requests
- issue tracking
- code review

Typical workflow:

`feature branch` → `pull request` → `review` → `merge`

Team Assignment Structure

Students work in **teams of two to three**.

Project setup:

- Each student creates an individual branch in the repository: `https://github.com/Zazazzzzz/Credit-Risk-Student-Workspace` (*send an email to be added as a collaborator*)
- Each team creates a shared project file and starts developing a script collaboratively
- Submit an initial test contribution (e.g., small script or file) to understand the collaboration workflow
- Use this setup throughout the course to jointly develop and submit the final assignment

Course Development Workflow

Initial Setup:

```
Install Python  →  Install PyCharm (IDE)  →
    Install Git  →  Create GitHub Account
```

After the setup:

```
Create Project → Write Code → Commit → Push →
                Pull Request
```

Objective: Develop models collaboratively while maintaining reproducible and well-structured research code.

Python: Programming

Core Python Libraries

Key tools used in this course:

NumPy

- vectorized computations
- random number generation

Pandas

- portfolio data handling

SciPy

- probability distributions

Matplotlib / Seaborn

- visualization of results

Programming Focus

The course emphasizes:

- Monte Carlo simulation design
- modular functions
- clear model implementation
- efficient numerical computation

The goal is to translate **financial models into executable code**.

First Portfolio Simulation

Before introducing theoretical models, we begin with a simple simulation.

Example portfolio:

- A bank lends **€100,000 each to 10 firms**.
- All firms promise to repay the full amount in one year (no interest, single payment).
- Each firm may default:
  - Firm 1: 1% chance of default
  - Firm 2: 2%
  - ...
  - Firm 10: 10%
- If a firm defaults, it only repays **€50,000**
  $\Rightarrow$ the bank loses €50,000

What Do We Want to Understand?

On average:

- How much loss should the bank expect?

In reality:

- Losses are uncertain
- If we repeat this situation **1,000 times (simulation)**, what does the **distribution of total losses** look like?

Simulation Workflow

Simulation steps:

1. simulate default indicators
2. compute losses for each obligor
3. aggregate portfolio losses
4. repeat for many scenarios
5. analyse the loss distribution

This introduces key concepts used later:

- random variables
- probability distributions
- tail risk

Motivation for Credit Risk Models

The simulation illustrates an important question:

How do we model dependence between defaults?

This motivates the models studied later:

- threshold models
- factor models
- copulas
- portfolio loss distributions